



UNIDAD 3 – SCRIPTS EN LINUX

1 CREACIÓN DE SCRIPTS EN LINUX:

2 Trabajo con el archivo de secuencias del shell:

Los shell de Linux, además de permitir el uso de comandos, permiten crear programas formados a base de comandos, llamados de muchas formas: shell-scripts, scripts, guiones, etc. Además de los comandos, en los scripts podemos utilizar muchas de las estructuras típicas de la programación tradicional: estructuras iterativas, estructuras de control, etc.

Un script, es esencialmente, un archivo de texto que contiene en cada línea un comando, una línea en blanco, o un comentario. Los comentarios comienzan por el carácter #, y se pueden incluir en cualquier parte de una línea de un script.

Hay que tener en cuenta dos cosas a la hora de hacer un script:

- 1- La primera línea debe contener lo siguiente: `#!/bin/sh`. Esto indica al shell que se trata de un fichero script y que debe encargarse su ejecución a `/bin/sh`, que es la ubicación del propio shell.
- 2- Para poder ejecutar un script es necesario que establezcamos previamente el permiso de ejecución del fichero correspondiente.

Los scripts, son simples ficheros de texto que podemos crearlos con cualquier editor como emacs o vi. La primera línea "`#!/bin/sh`" identifica el fichero y le dice al intérprete de comandos como ejecutarlo.

Para probar un archivo de secuencias del shell comprobando todos sus pasos, según se desarrollan:

```
sh -x nombre_script
```

- ❑ echo: Se utiliza para mostrar un mensaje. Poniendo toda una secuencia de caracteres entre comillas, permite mostrarlo toda:

EJ: echo "Por favor espere "
 echo hola
 echo `Por favor, espere....` > msg



- ❑ Comentario: # : El comentario es una anotación para el usuario. El shell ignora un comentario

```
EJ:  #Autor: -----
      # Programa: -----
      echo "Comprobamos quien está conectado"
      who # Quien está conectado
      cal
      date
      echo Fin
```

- ❑ Comillas en Linux:
 4. (") Comillas dobles: Acceder al contenido de una variable.
 Ej: v="texto"
 n="El valor de v es \$v"
 echo \$n: El valor de v es texto
 5. (') Comillas simples: El shell no interpreta las variables. Si se aplica en el caso anterior el mensaje sería → El valor de v es \$v.
 6. (`) Comillas invertidas: El shell ejecuta la línea contenida entre comillas.
 Ej: v=6
 var=`expr \$v + 2`
 echo \$var : 8

3 Variables en los programas de shell:

Para usar variables, debemos saber cómo asignar un valor a una variable y la forma de acceder después al valor asignado a la variable. Existen cuatro formas de asignar valor a una variable.

- 1.- Utilizando asignaciones directas.
- 2.- Utilizando la orden read.
- 3.- Utilizando los parámetros de la línea de orden.
- 4.- Sustituyendo la salida de una orden.

4 Asignación directa:

nombre_variable=valor_variable

- ❑ No incluye espacios a los lados del signo igual.
- ❑ Si valor_variable contiene espacios en blanco debe ir entre comillas.

```
Ej:     directorio=/home/usuario/seguridad
       cp archivo $directorio
```

- ❑ El shell sustituye \$directorio por su valor.



5 Uso de la orden read:

La orden read toma la línea siguiente de entrada y le asigna a una variable.

Ej: # Nombre
 # Vamos a copiar un archivo especificado a un directorio.
 directorio=/home/usuario/seguridad
 read archivo
 cp \$archivo \$directorio

La orden read detiene la orden y espera la entrada procedente del teclado

6 Uso de parámetros de línea de órdenes:

Los elementos de la línea de órdenes son las secuencias de caracteres separados por caracteres de espacios en blanco o tabulación. (Utilice comillas para señalar un conjunto de caracteres separados por espacios que representen un elemento).

Las variables de acceso a los parámetros son:

Variable	Significado
\$1	Primer parámetro pasado
\$2	Segundo parámetro pasado
\$3	Tercer parámetro pasado

\${10}	Décimo parámetro pasado
\${11}	Undécimo parámetro pasado

\$@	Todos los parámetros pasados como argumento
\$0	Nombre del archivo ejecutado (ruta completa)
\$#	Número de parámetros pasados
\$\$	(PID) del proceso que se está ejecutando
\$?	Código de retorno de la última orden ejecutada (0 → éxito, 1 → erróneo)

Para utilizar los parámetros de más de dos cifras numéricas, tenemos que encerrar entre llaves el número.

El comando **shift** nos permite desplazar una posición el resto de parámetros y desechar el parámetro primero, de manera que \$2 pasa a ocupar la posición \$1, \$3 la de \$2 y así sucesivamente. Esto nos permite utilizar los parámetros con las estructuras de control iterativas.

7 Sustitución de una salida de una orden:

Puede asignar a una variable el resultado de una orden ejecutada.

Ej: `pwd`

La orden de trabajo del directorio, está puesta entre acentos graves en lugar de apóstrofes.

Ej: Modificar el nombre de un archivo añadiendo el mes, día y año actual al nombre del archivo.

#Asignar td a la fecha actual en forma mmddyy

td=`+%m%d%y`

#Asignar nuevo nombre al archivo



mv \$1 \$1.\$td

8 Programación con estructuras controladas:**9 Estructuras condicionales:**

- 10 **CASE:** La estructura case es una estructura de decisión que le permite seleccionar una entre varias posibilidades de acción según el valor de una variable.

Sintaxis:

```
case valor in
    patron) instruccion(s);;
    patron) instruccion(s);;
    -----
esac
```

El parámetro valor se compara con cada uno de los patrones empezando por el primero. Las instrucciones que se ejecutan si valor coincide con un patrón, se terminan con dos puntos y coma.

Ej:

```
#Permitir al usuario elegir entre varias opciones
#Visualización del menú
echo "Elija entre V,B o S"
echo "[V]isualiza un fichero"
echo "[B]orrar un fichero"
echo "[S]alir"
#Obtiene respuesta
read opcion
case $opcion in
    V|v) more archivo;;
    B|b) rm archivo;;
    *) echo saliendo;;
esac
```

La barra vertical (|) se utiliza para proporcionar una elección.
El patrón * para representar el resto de valores

- 11 **IF:** Estructura de decisión que permite seleccionar entre dos posibilidades de acción.

Sintaxis:

```
if condicion; then
    acción1
else
    acción2
fi

if condicion1; then acción1
elif condicion2; then acción2
else
    acción3
fi
```

#Sólo se utiliza un fi al final de toda la estructura.

```
Ej:
#Leer valor y si el valor introducido es uno, escribir "el valor es uno", si es dos, escribir
"el valor es dos"
read var
if [ $var = 1 ]; then
    echo " El valor es uno"
elif [ $var = 2 ]; then
    echo "El valor es dos"
fi
```

Las palabras reservadas if, then, else, elif, fi, deben ser las primeras en una línea o estar situadas después de un ; indicando que se trata de otra línea.

12 Otras formas más simples de ejecutar comando condicionalmente serían:

- ❑ comando1 && comando2 : Comando2 se ejecuta si la ejecución de comando1 tiene éxito.
- ❑ Comando1 || comando2: Comando2 se ejecuta si la ejecución de comando1 no tiene éxito.

13 Estructuras iterativas:

Las instrucciones **break** y **continue** alternan la ejecución de un bucle. Colocadas en su interior sirven para:

- **break**: Romper su ejecución y saltar fuera del bucle.
- **continue**: Comenzar una nueva iteración.

14

- **FOR:**

El bucle for va a permitir especificar un conjunto de archivos o valores para que se utilicen con algunas órdenes.

Sintaxis:

```
for v in conjunto
do
    Sentencia(s)
done
```

Ejemplo:

```
for v in 1 2 3 4
do
    Echo " El valor de v es ... $v"
done
```

- **WHILE:**

Realiza la iteración mientras se cumpla la condición.

Sintaxis:

```
while condicion
do
    Sentencia(s)
done
```

Ejemplo:



```
#Introducir números hasta pulsar el 0.
read valor
while [ valor != 0 ]
do
    echo "El valor introducido es $valor"
    read valor
done
echo "Hemos finalizado con el valor $valor"
```

- **UNTIL:**

Los bucles con la instrucción until se escriben con el mismo formato que con while, excepto que la condición a formular, tras la palabra until es de finalización del bucle (hasta que se cumpla la condición).

15 Ordenes:

- **Orden test:** Evalúa una expresión y emite una salida verdadera o falsa. Suele utilizarse en combinación con otras órdenes como if. test sirve para comparar expresiones numéricas, cadenas de caracteres y para evaluar distintas características de un fichero.

Su sintaxis es: *test [expresión]*

Donde expresión puede ser:

Expresión

-r fichero
-w fichero
-x fichero
-f fichero
-d fichero
-c fichero
-b fichero
-s fichero

Evalúa si el fichero existe y:

se puede leer.
se puede escribir
se puede ejecutar
es un fichero normal (no es un directorio)
es un directorio
es un fichero especial de tipo carácter
es un fichero especial de tipo bloque
tiene un tamaño mayor que cero.

Expresión

-z s1
-n s1
s1 = s2
s1 != s2
s1
n1 -eq n2
n1 -ne n2
n1 -gt n2
n1 -lt n2
n1 -ge n2
n1 -le n2

Evalúa si:

la longitud de la cadena s1 es cero
la longitud de la cadena s1 no es cero
las dos cadenas son iguales
las dos cadenas son distintas
la cadena s1 existe
n1 y n2 tienen el mismo valor numérico
n1 y n2 tienen distinto valor numérico
n1 tiene un valor mayor que n2
n1 tiene un valor menor que n2
n1 tiene un valor mayor o igual que n2
n1 tiene un valor menor o igual que n2

Tanto s1 como s2 pueden ser variables o cadenas encerradas entre comillas, asimismo, n1 y n2 pueden ser variables o valores numéricos enteros.

Se pueden combinar o unir varias condiciones mediante:

- 1.- **!**: operador lógico not.
- 2.- **-a**: operador lógico and.



3.- **-o:** operador lógico or.

Ejemplo1:

```
if test -f fichero; then
    echo "El fichero existe y es normal"
fi
```

Ejemplo2:

```
#Buenos días si la hora es menor que 12
#Buenas tardes si la hora es menor que 17
#Buenas noches si la hora es mayor que 16
```

```
#Obtener la hora
hora=`date +%H`
#Comprobar
if test $hora -lt 12; then
    echo "Buenos días"
elif test $hora -lt 17; then
    echo "Buenas tardes"
else
    echo "Buenas noches"
fi
#Sólo se utiliza un fi al final de toda la estructura
```

- **Orden expr:** Realiza operaciones sobre variables o valores o evalúa expresiones y muestra su resultado sobre la salida estándar.

Su formato es: `expr Expresiones`

Ejemplo:

```
z=2
v=`expr $z + 10`
echo $v
```

Algunas expresiones que se pueden formular son las siguientes:

- Expresión operador-relacional expresión: El resultado de la comparación se muestra por la salida estándar; pero redirigiendo la salida, podemos almacenarlo en una variable para su uso posterior.
 2. = igual
 3. \> mayor
 4. \>= mayor o igual
 5. \< menor
 6. \<= menor o igual
 7. != distinto
- Expresión operador-aritmético expresión: El resultado es numérico y el valor de la expresión también.
 9. + suma
 10. - resta
 11. / división
 12. * producto
 13. % módulo



- 16 **Funciones:** Las funciones de shell son similares a los procesos de usuario. Las funciones se utilizan para realizar pequeñas operaciones, ya que se guardan en memoria y su depuración es más laboriosa.

```
function nombre_funcion()
{
    lista comandos
}
```

Hay que tener en cuenta que para utilizar una función, esta ha de estar definida de antemano. Para enviar parámetros a una función aplicamos: *nombre_función lista parámetros*. Dentro de la función los parámetros se recogen en variables especiales del tipo \$1, \$2, etc, cuyo funcionamiento es similar a las utilizadas a nivel general en cualquier script.

return → Fuerza la terminación de la ejecución de un procedimiento o función

■ **Funciones incluidas dentro del mismo shell:**

```
#Zona de definición de funciones
function evalua()
{
    #Si cumple la condición muestra el mensaje
    if [ $2 -gt 20 ]; then
        Echo "$1 tienes más de 20 años"
    fi
    return
}

#Guión del shell
read nombre edad
# Le pasa unos valores a la función como parámetros
evalua $nombre $edad
```

■ **Declarar una variable local:**

```
#Zona de definición de funciones
function evalua()
{
    #declaramos una variable local
    local var1=10
    echo "Valor de la variable"
}

#Guión del shell
read nombre
#Le pasa un valor a una función como parámetro
evalua $nombre
echo "El valor de la variable local de la función es: $var1 " #Aparece en blanco
```

Ejercicio: Probar el funcionamiento, pero esta vez sin declarar la variable var1 como local.

■ **Devolver un valor a un procedimiento desde una función:**



#La función recibe un valor y si se cumple la condición devuelve el valor 1000, de lo contrario devuelve el 500. En el procedimiento le sumamos 10 al valor recibido.

```
Function f()
{
    k=1000
    #Con la instrucción echo, devolvemos valores al procedimientos
    if [ $1 -gt 4 ]; then
        echo $k
    else
        echo 500
    fi
}

#Guión del shell
read var1
#Ejecutamos la función, almacenando el valor recibido en la variable e
e=`f $var1`
j=`expr $e + 10`
echo $j
```

■ **Funciones almacenadas en otro archivo:**

```
#Declarar las funciones en otro archivo llamado funciones

#Contenido del guión que accede a la función.
./usuario/funciones # Ejecutamos el guión de las funciones
read nombre
#Le pasa un valor a la función como parámetro
evalua $nombre
```

Por lo tanto, tendremos dos archivos, uno con las funciones declaradas y otro que será el guión que acceda a ellas.